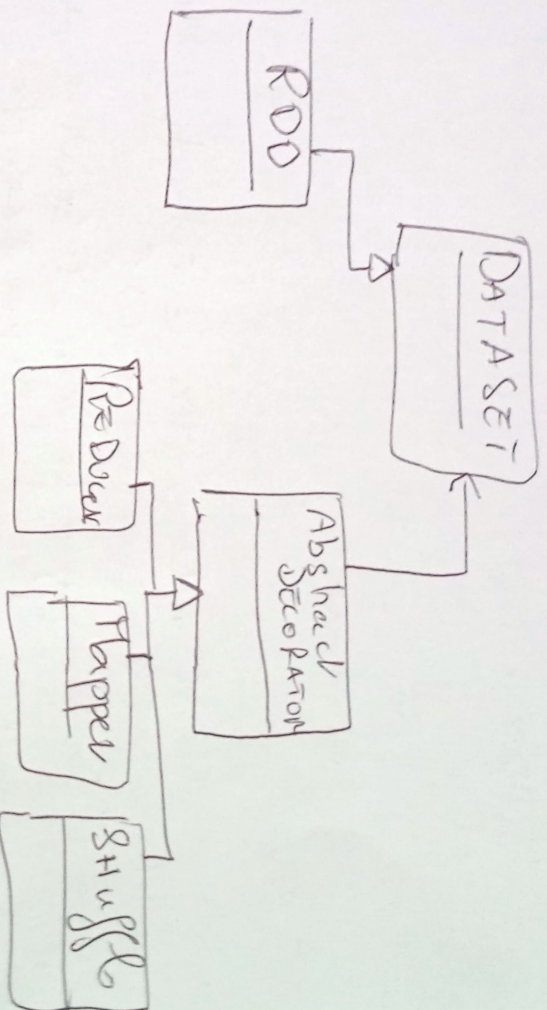


Question 1 cette écriture d'instruction est uniquement valable avec le Pattern Decorator qu'on a étudié. avec :



Problème

1) Modèle attaquant : Chain of Responsibility de suffixe de répétition le code du cours ;

public abstract class MultiStepsAttack {

private MultiStepsAttack attackStep;

public void setNextAttack (MultiStepsAttack attackStep)

{ this.attackStep = attackStep; }

public MultiStepsAttack getAttackStep ()

{ this.attackStep = attackStep; }

public abstract void triggerAttack () ; }

1) pour l'ensemble l'ensemble :

public class FirstAttack extends MultiStepsAttack {

public void triggerAttack () {

sysol ("collecte d'informations l'ensemble l'ensemble")

if (getAttackStep () != null) getAttackStep ().triggerAttack ();

else sysol ("Fin de l'attaque")

} // fin de l'ensemble

} // fin de la classe

}

[3]

2^e getMinnar Cancel

```
public class SadMinAttack extends MultiStepAttack {  
    public void triggerAttack() {  
        Sys0("Je suis un worm qui infecte les pages web et  
            les serveurs Sslais")  
        if (getAttackStep() != null) getAttackStep().triggerAttack();  
        else Sys0("fin de l'attaque");  
    } // fin Method  
}  
// fin class.
```

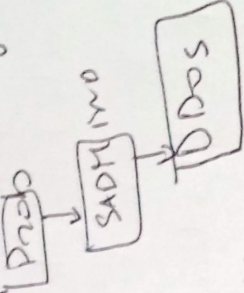
3^e Execution Cancel

```
public class DDOSAttack extends MultiStepAttack {  
    public void triggerAttack() {  
        Sys0("Je provoque le Deni de service par  
            inondation par exemple...")  
        if (getAttackStep() != null) getAttackStep().triggerAttack();  
        else Sys0("fin de l'attaque");  
    } // fin Method  
}  
// fin class.
```

— close Deviser de l'attaque Multi-Étape:

```
public class Devis {  
    main() {  
        MultiStepAttack probe = new ProbingAttack();  
        MultiStepAttack sadMin = new SadMinAttack();  
        MultiStepAttack Ddos = new DDOSAttack();  
        probe.setNextAttack(sadMin);  
        sadMin.setNextAttack(Ddos);  
        probe.triggerAttack();  
    } // fin main  
}  
// fin class Devis
```

2) Nodes s'analyse / Detection: le seul Pattern qui s'apparente aux graphes d'attaques, et qu'on a étudié en le Composite. Il suffit d'appliquer l'exemple du cours. Soit par exemple le graphe



1) L'interface:

```
public interface
```

```
AttackGraph {
```

```
public void Analyse(); }
```

2) On a une seule feuille, DDoS.

```
public class DDoS implements AttackGraph
```

```
{ private String IPAdress;
```

```
private String Date;
```

```
public DDoS (String IP, String Date)
```

```
{ IPAdress = IP;
```

```
Date = date; }
```

```
public void Analyse ()
```

```
{ Syso ("IPAdress" + "Date"); }
```

3) Classe Composite:

```
public class AttackStep implements AttackGraph
```

```
{ private List <AttackGraph> attackSteps;
```

```
= new ArrayList <AttackGraph>;
```

```
public String IPAdress;
```

```
private String Date;
```

```

public AttackSteps (String IP, String date)
{
    IPAddress = IP;
    Date = date;
}

public void Analyse () {
    for (AttackGraph ag : AttackSteps)
        ag.Analyse ();
    public void addAttackStep (AttackGraph ag)
    {
        AttackSteps.add (ag);
    }
    public void removeAttackStep (AttackGraph ag)
    {
        AttackSteps.remove (ag);
    }
}

③ AttackGraph Java Class
public class Java Analyse {
    main () {

```

```

        DDOS dos = new DDOS ("192.168.1.7", "23/3/2022")
        AttackStep sashinda = new AttackStep ("192.168.1.8", "24/3/2022")
        # sashinda.addAttackStep (dos)
        AttackStep probe = new AttackStep ("192.168.2.2", "25/3/2022")
        probe.addAttackStep (sashinda);
        probe.Analyse ();
    }
}

```